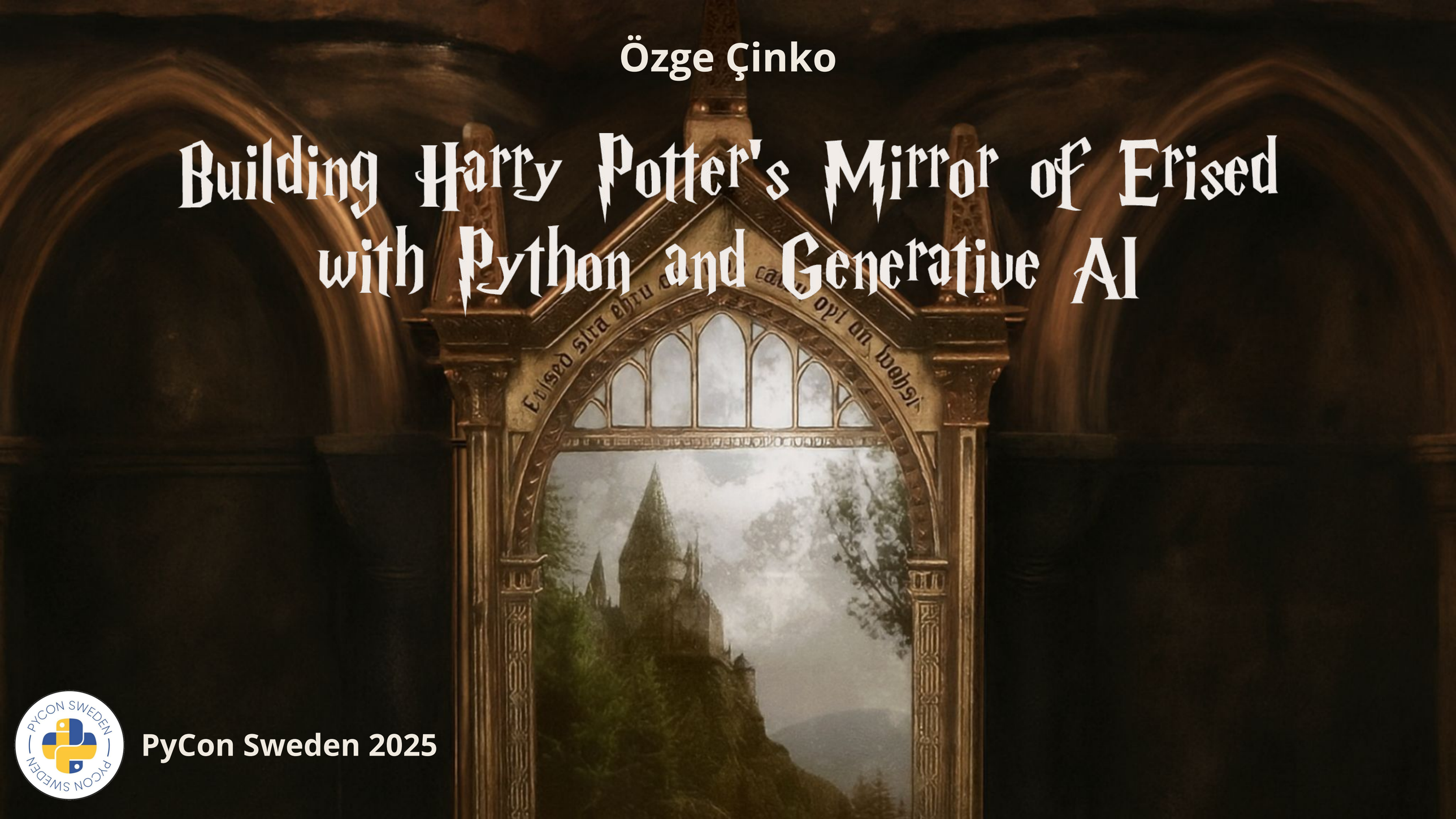
Özge Çinko

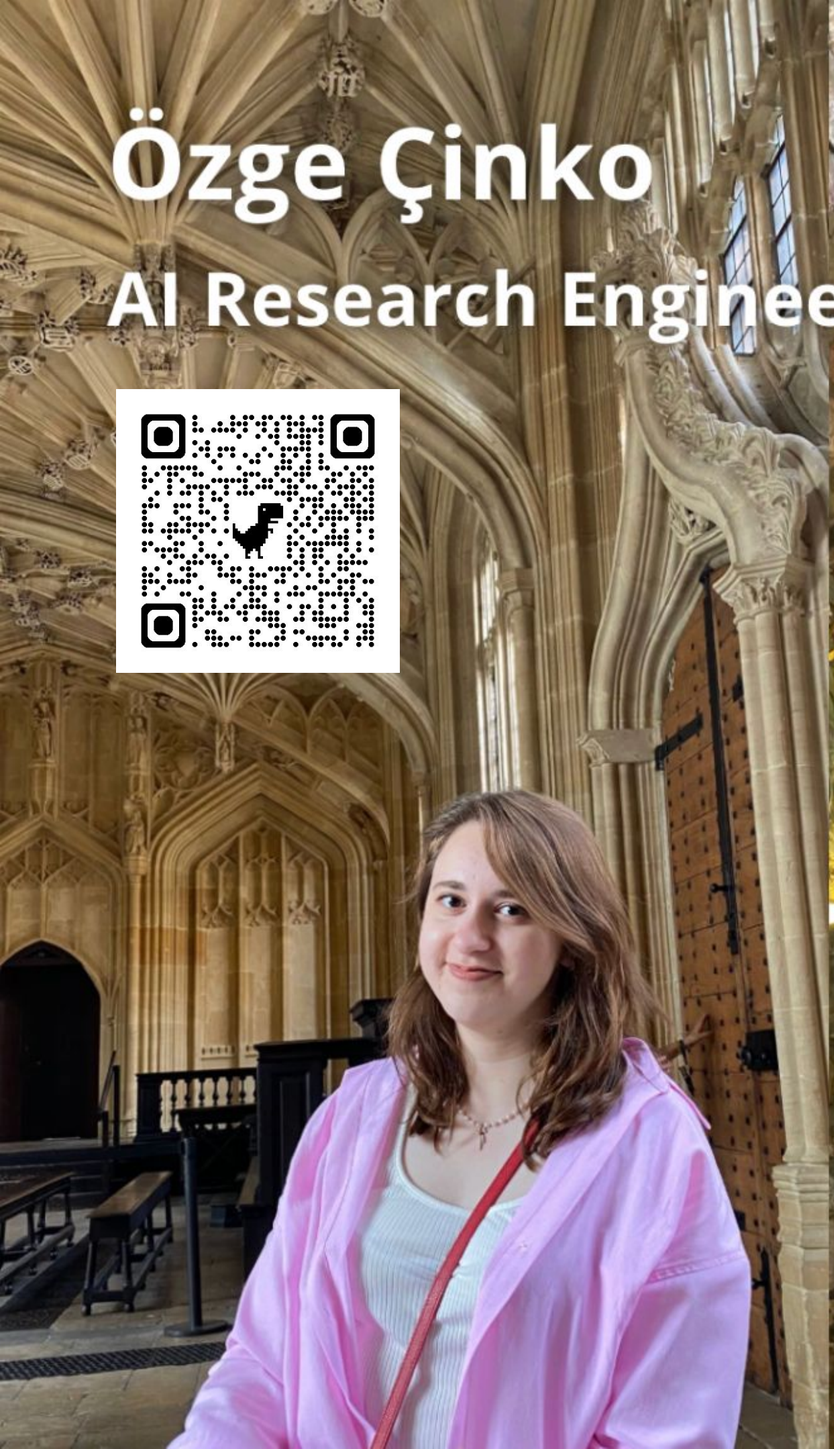
Building Harry Potter's Mirror of Erised with Python and Generative AI



PyCon Sweden 2025

Özge Çinko

AI Research Engineer @ Huawei



Contents

- What is Mirror of Erised?
- Let’s Build The Magic!
- System Architecture
- Technology Stack
- From Words to Meaning
- From Meaning to Vision
- Web Wizardry
- Lessons Learned
- Challenges To You
- Question & Answer

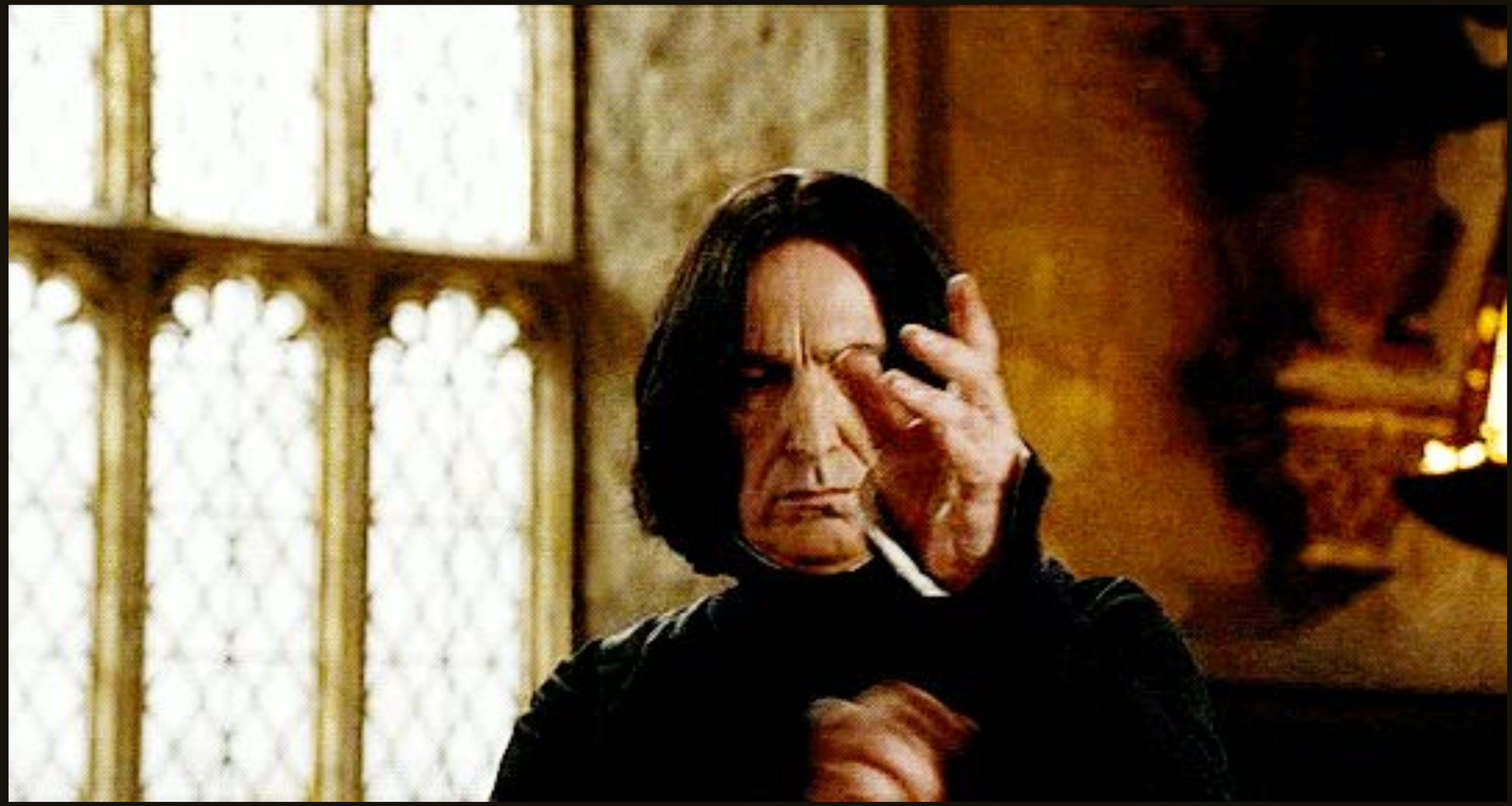
What is Mirror of Erised?

The Mirror of Erised shows not your face... but your heart’s deepest desire.

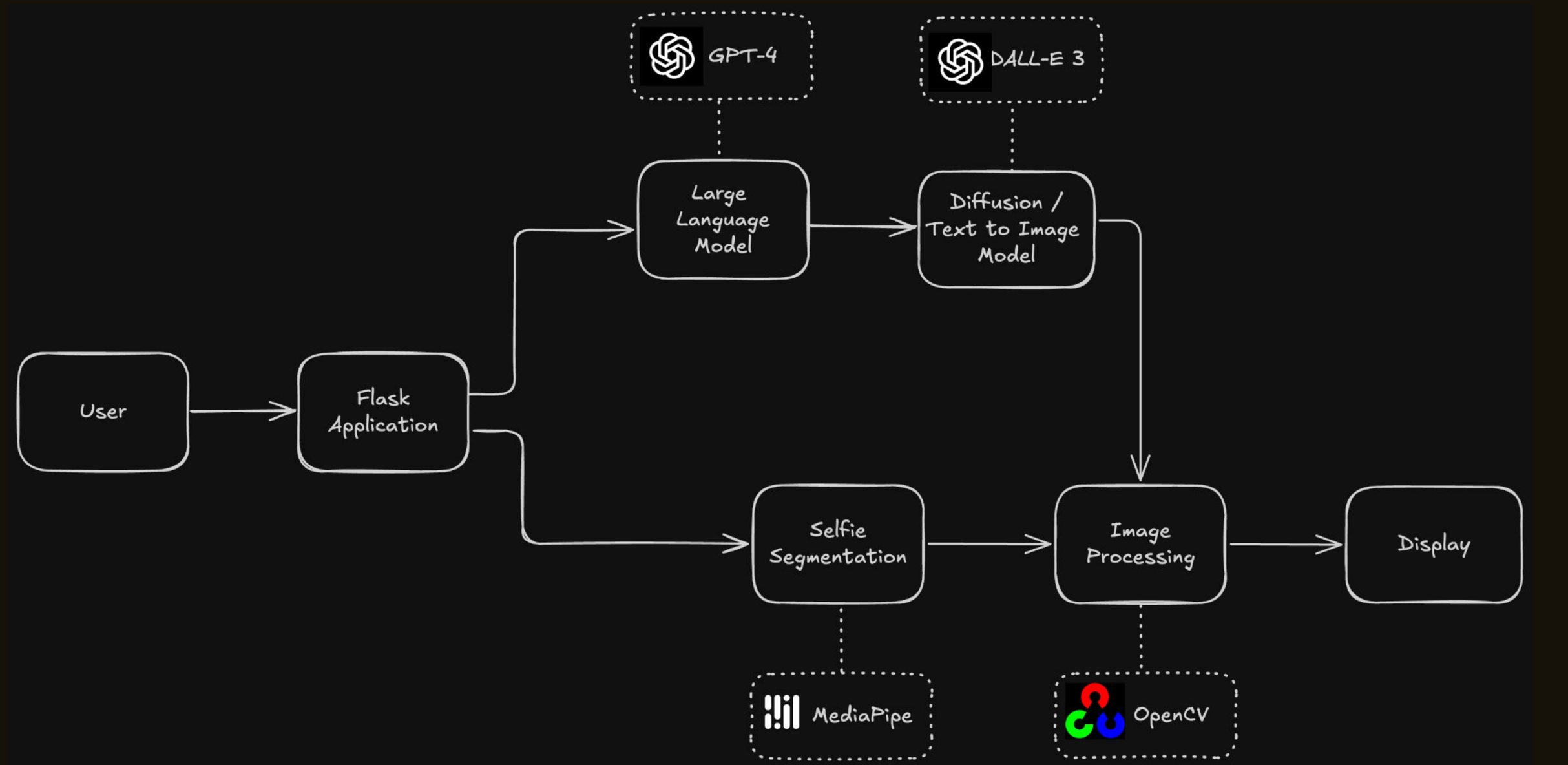
Mirror of Erised Scene

When The Magic Happens...

Let’s Build The Magic!



System Architecture



Technology Stack

Python

Flask

OpenAI API
GPT-4 & DALL-E 3

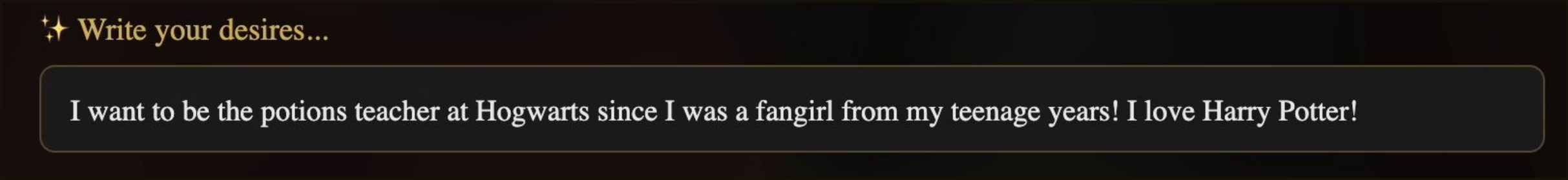
MediaPipe

OpenCV

From Words to Meaning

User Prompt is The Key

- **Python can't read minds. We need this input to understand what's in the user's heart.**



From Words to Meaning

Why We Need The User’s Words?

- My ambitious plan: webcam and social media data = mind reading!
- Reality: GDPR says NOOOOO!
- The secure and ethical solution: prompt engineering, because Python can't perform legilimens.

From Words to Meaning

System Prompt: Telling GPT What To Do

```
def generate_visual_prompt(user_text):
    system_prompt = """You are an AI emotion interpreter.

    Your task is to read a personal text written by a user about their
    deepest desire or dream,
    and transform it into a clear and emotionally rich visual
    description suitable for text-to-image generation.

    Steps:

    1. Analyze the text to identify:
        - The main emotions (e.g., nostalgia, hope, peace, love,
        ambition)
        - The key symbols or imagery that appear or could represent
        these emotions
        - The general atmosphere or mood (e.g., calm, vibrant, surreal)
    2. Combine these elements into a single scene description that can
    be visualized by an image model.
        - Avoid abstract language; describe concrete visual details
        (setting, light, colors, objects, people, actions)
        - Do not mention text, words, or letters in the image.

    Output format (JSON only):
    {
        "emotions": ["emotion1", "emotion2"],
        "symbols": ["symbol1", "symbol2"],
        "scene": "detailed visual scene description here"
    }

    Now analyze this text:"""
```

- **Writing a good prompt is <u>essential</u> for generating an accurate visual prompt.**

From Words to Meaning

GPT API Call

```python
def generate_visual_prompt(user_text):
    ...
    try:
        response = client.chat.completions.create(
            model="gpt-4",
            messages=[
                {"role": "system", "content": system_prompt},
                {"role": "user", "content": user_text}
            ],
            temperature=0.8,
            max_tokens=500
        )

        llm_output = response.choices[0].message.content.strip()
        return parse_llm_output(llm_output)

    except Exception as e:
        print(f"GPT Error: {e}")
        return get_fallback_response(user_text)
      ...
```

- **temperature: 0.8**
- **max_tokens: 500**

From Words to Meaning

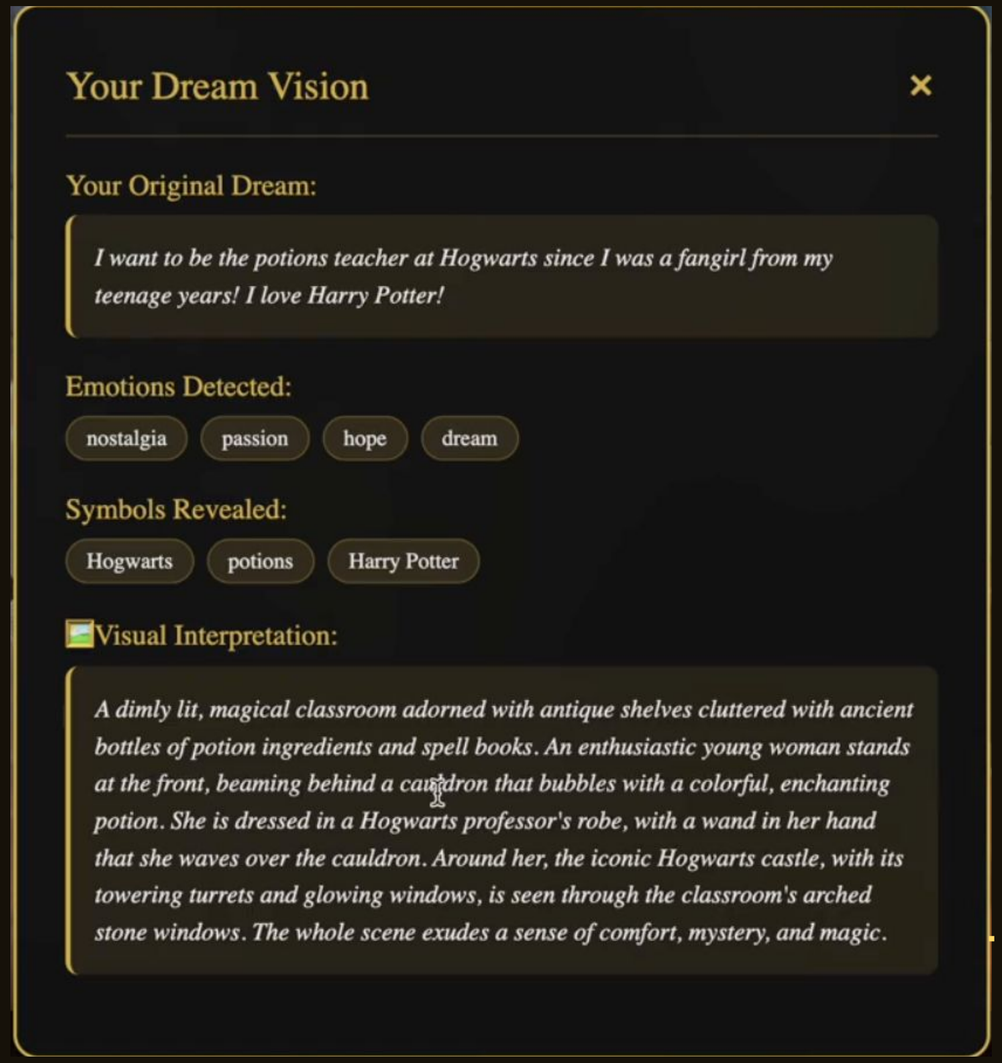
When GPT Forgets JSON

```
def generate_visual_prompt(user_text):
    ...
        llm_output = response.choices[0].message.content.strip()

        print("GPT RAW OUTPUT")
        print(llm_output)

        # Extract JSON
        json_match = re.search(r'\{.*\}', llm_output, re.DOTALL)
        if json_match:
            try:
                visual_data = json.loads(json_match.group())
            except:
                # Manual parsing if JSON fails
                emotions_match = re.search(r'"emotions":\s*\[(.*?)\]',
                                           llm_output)
                symbols_match = re.search(r'"symbols":\s*\[(.*?)\]',
                                          llm_output)
                scene_match = re.search(r'"scene":\s*"([^"]*)"',
                                        llm_output)

                visual_data = {
                    "emotions": ["hope", "dream"] if not emotions_match
                  else [e.strip().strip('"') for e in
                        emotions_match.group(1).split(',')],
                    "symbols": ["light", "future"] if not symbols_match
                  else [s.strip().strip('"') for s in
                        symbols_match.group(1).split(',')],
                    "scene": f"Beautiful scene representing:
                  {user_text}" if not scene_match else
                  scene_match.group(1)
                }
      ...
```

- **When GPT forgets proper JSON syntax, we use regex to parse and extract the data anyway.**

From Words to Meaning

GPT Creates Visual Prompt



Visual Interpretation:
A dimly lit, magical classroom adorned with antique shelves cluttered with ancient bottles of potion ingredients and spell books. An enthusiastic young woman stands at the front, beaming behind a cauldron that bubbles with a colorful, enchanting potion. She is dressed in a Hogwarts professor's robe, with a wand in her hand that she waves over the cauldron. Around her, the iconic Hogwarts castle, with its towering turrets and glowing windows, is seen through the classroom's arched stone windows. The whole scene exudes a sense of comfort, mystery, and magic.

From Meaning to Vision

Transforming Words Into Pixels

```python
def generate_image_dalle3(prompt):
    try:
        response = client.images.generate(
            model="dall-e-3",
            prompt=prompt,
            size="1024×1024",
            quality="standard",
            n=1
        )

        image_url = response.data[0].url
        return download_image(image_url)

    except Exception as e:
        return None
```

- **DALL-E 3’s moment!**

From Meaning to Vision

The Digital Image Pipeline

```python
def download_image(image_url):
    """Converting web URL to computer-readable format"""
    try:
        image_response = requests.get(image_url)
        image_array = np.frombuffer(image_response.content, np.uint8)
        # Convert to OpenCV format (BGR)
        image = cv2.imdecode(image_array, cv2.IMREAD_COLOR)
        image = cv2.resize(image, (1280, 720))
        return image

    except Exception as e:
        return None
```

- **DALL-E gives us a web URL, we need an OpenCV image.**
- **OpenCV uses BGR color format (not standard RGB).**
- **Our webcam is 1280x720, the background must match exactly.**

From Meaning to Vision

Fallback System: Local Image Generation

```python
def generate_image_local(prompt):
    """When DALL·E is busy saving the world, we handle it ourselves"""
    height, width = 720, 1280
    background = np.zeros((height, width, 3), dtype=np.uint8)

    # Create magical gradient background
    for i in range(height):
        r = int(100 + (i / height) * 100)  # Red gradient
        g = int(150 + (i / height) * 50)   # Green gradient
        b = int(200 + (i / height) * 55)   # Blue gradient
        background[i, :] = [b, g, r]  # OpenCV uses BGR!

    return background
```

- **What if OpenAI API is down?**

From Meaning to Vision

Image Generation

```python
def generate_image(prompt):
    """Intelligent decision maker for image generation"""
    image = generate_image_dalle3(prompt)

    if image is not None:
        return image
    return generate_image_local(prompt)
```

From Meaning to Vision

Background Management

```python
def update_background(new_image, user_text, visual_prompt, emotions,
symbols):
    """Update the mirror with new dream vision"""
    global current_background, background_history
    current_background = new_image

    # Save to history (last 10 dreams)
    background_history.append({
        'timestamp': datetime.now().isoformat(),
        'dream': user_text,
        'prompt': visual_prompt,
        'emotions': emotions,
        'symbols': symbols
    })

    if len(background_history) > 10:
        background_history.pop(0)

    save_path = "static/img/current_background.jpg"
    cv2.imwrite(save_path, current_background)
```

Web Wizardry

Webcam Setup

```python
def generate_frames():
    """The mirror's eye - capturing real world"""
    cap = cv2.VideoCapture(0)
    cap.set(cv2.CAP_PROP_FRAME_WIDTH, 1280)
    cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 720)

    while True:
        success, frame = cap.read()
        if not success:
            break

        # Mirror effect - flip horizontally
        frame = cv2.flip(frame, 1)

        yield process_frame(frame)
```

Web Wizardry

Segmentation Magic

```python
# MediaPipe setup
mp_selfie_segmentation = mp.solutions.selfie_segmentation
segment = mp_selfie_segmentation.SelfieSegmentation(model_selection=1)

def process_frame(frame):
    """Separate person from background like magic!"""
    rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    # create segmentation mask
    results = segment.process(rgb_frame)

    if results.segmentation_mask is not None:
        mask = results.segmentation_mask
        condition = mask > 0.6
        return condition
```

- **This is where the real wizardry happens.**

Web Wizardry

Background Replacement

```python
def generate_frames():
    while True:
        success, frame = cap.read()
        frame = cv2.flip(frame, 1)

        # Get segmentation mask
        results = segment.process(cv2.cvtColor(frame,
                                               cv2.COLOR_BGR2RGB))

        if results.segmentation_mask is not None:
            mask = results.segmentation_mask
            condition = mask > 0.6

            # Resize AI-generated background to match webcam
            bg_resized = cv2.resize(current_background,
                                    (frame.shape[1], frame.shape[0]))

            # The final magic: replace background
            output = np.where(condition[:, :, None], frame, bg_resized)
        else:
            output = frame

        # Convert to bytes for web streaming
        ret, buffer = cv2.imencode('.jpg', output,
                                   [cv2.IMWRITE_JPEG_QUALITY, 80])
        yield buffer.tobytes()
```

- **All this happens 30+ times per second..**

Web Wizardry

Flask Routes: Platform 9¾ For Our Magic

```python
@app.route('/')
def index():
    """The main mirror interface"""
    return render_template('index.html')

@app.route('/video_feed')
def video_feed():
    """Live video streaming endpoint"""
    return Response(generate_frames(),
                    mimetype='multipart/x-mixed-replace;
boundary=frame')

@app.route('/generate_dream', methods=['POST'])
def generate_dream():
    """Where dreams become reality"""
    user_text = request.json.get('dream', '').strip()

    if not user_text:
        return jsonify({'success': False,
                        'error': 'The mirror awaits your dream...'})

    # The magic pipeline
    llm_result = generate_visual_prompt(user_text)
    generated_image = generate_image(llm_result['visual_prompt'])

    return jsonify({'success': True,
                    'message': 'Your dream has been visualized! 🎉'})
```

/ - The Mirror Interface

- **Serves the main HTML page where magic happens**

/video_feed - Live Vision Stream

- **Streams real-time video with background replacement**

/generate_dream - Dream Factory

- **Accepts user dreams → GPT analysis → DALL-E generation**

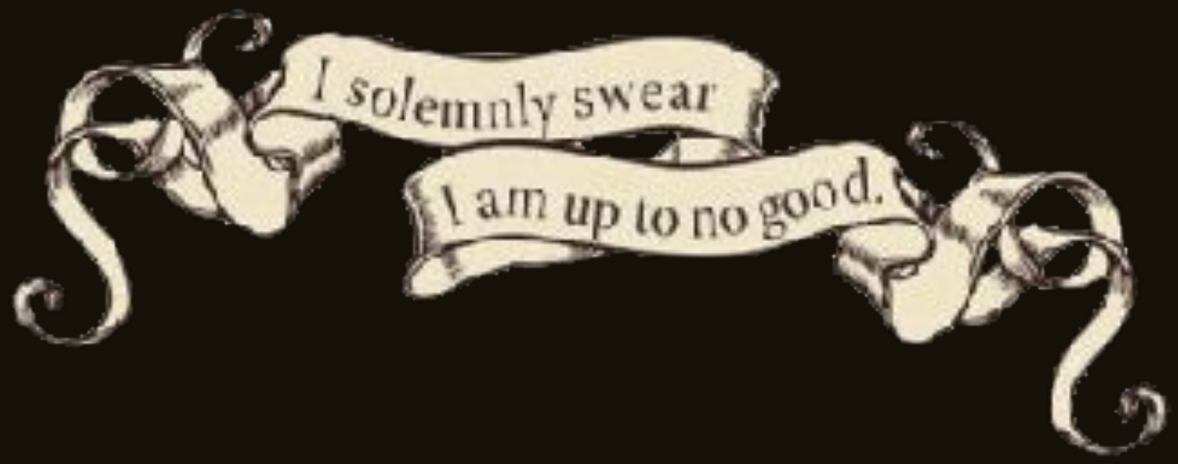
Web Wizardry

Building the Magical Interface


Mirror of Erised
Here’s The Magic
Mission Completed!
Your Dream Vision
Your Original Dream:
I want to be the potions teacher at Hogwarts since I was a fangirl from my teenage years! I love Harry Potter!
Emotions Detected:
nostalgia
passion
hope
dream
Symbols Revealed:
Hogwarts
potions
Harry Potter
Visual Interpretation:
A dimly lit, magical classroom adorned with antique shelves cluttered with ancient bottles of potion ingredients and spell books. An enthusiastic young woman stands at the front, beaming behind a cauldron that bubbles with a colorful, enchanting potion. She is dressed in a Hogwarts professor's robe, with a wand in her hand that she waves over the cauldron. Around her, the iconic Hogwarts castle, with its towering turrets and glowing windows, is seen through the classroom's arched stone windows. The whole scene exudes a sense of comfort, mystery, and magic.
✨ Write your desires...
Whisper your desires to the mirror...
Reset
Generate
Mirror my desires...

Lessons Learned

What Mirror Taught Us



- Prompt engineering is crucial for good results.
- Python ecosystem makes AI accessible to everyone.

Challenges To You

Your Turn to Create Magic!

- Analyze real-time facial expressions from the video feed and add those emotions to the visual prompt.
- Build a local version using open-source models (like Llama or Stable Diffusion).
- Create a dream history gallery where users can save and revisit their past desires!
- What if multiple people look into the mirror together? Generate a combined dream vision!

Any Questions?